

Doc. no. P0618R0
Date: 2017-03-02
Project: Programming Language C++
Audience: Library Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Deprecating `<codecvt>`

Table of Contents

1. [Revision History](#)
 - [Revision 0](#)
2. [1 Introduction](#)
3. [2 Stating the problem](#)
4. [3 Propose Solution](#)
5. [4 Other Directions](#)
6. [5 Formal Wording](#)

Revision History

Revision 0

Original version of the paper.

1 Introduction

This paper aims to address NB comment: GB-57, which was tentatively reviewed and accepted by LEWG at Issaquah 2016 (check wiki minutes to verify claim).

2 Stating the problem

As per GB-57:

The contents of are underspecified, and will take a reasonable amount of work to identify and correct all of the issues. There appears to be a general feeling that this is not the best way to address unicode transcoding in the first place, and this library component should be retired to Annex D, along side , until a suitable replacement is standardized.

Reviewing the NB comments, Beman Dawes (who provided a review and wording for updating our Unicode standard reference) gave additional feedback:

"I'm in favor of this proposed change, but won't be in Kona so Marshall asked me to post my views on the reflector. Here is some additional rationale of deprecation."

- As the Unicode standard and security experts point out, ill-formed UTF can be and has been used as an attack vector. Yet the `<codecvt>` facets don't provide the safe forms of error handling as their defaults, and it is far to hard to use them in a safe way.
- I've wanted to use `<codecvt>` in production code three or four times where turned out to not be suitable, but because the spec is so obscure, it took far too much time to realize that.
- In the couple of cases where `<codecvt>` would have been suitable, I couldn't use it because the code had to be portable, yet `libstdc++` didn't support until very recently. That, coupled with the fact that is a new header, means there just aren't a lot of users.
- In published usage surveys like BuiltWith, use of formerly popular non-UTF encodings like Shift-JIS and Big5 is plummeting compared to UTF encodings. The standard library badly needs modern C++ UTF conversion facilities, yet `<codecvt>` is an embarrassment. Let's deprecate it to clear the path for the future.

3 Proposed Solution

Move the whole of clause 22.5 [locale.stdcvt] to Annex D, updating the stable reference tags with a **depr.** prefix.

Deliberately retain the dated reference to ISO 10646, rather than relying on the dated reference in 1.2 [intro.refs], as a

concession to the deprecated material retaining references to UCS-2.

Review by the Library Working Group in Kona (2017) recommended that the `wstring_convert` and `wbuffer_convert` facilities from `<locale>` be deprecated at the same time.

4 Other Directions

The prospect of filing 30-40 defect reports and trying to beat this clause into reasonably well-specified and correct behavior is unappealing, but potentially vialble way to make progress.

Note that deprecation does not prohibit filing and fixing issues, but is is likely to reduce the priority given to such issues in the limited LWG time.

5 Formal Wording

These changes are relative to [N4640](#) in the pre-Kona mailing.

Editor's note: the change for the draft document is almost entirely a cut and paste of the relevant clauses. The only differences are new stable names, some introductory text for the purposes of deprecataion, and D.Xp1 is moved up from D.X.1p1, in order to serve as the introductory text.

22.1 General [localization.general]

- 2. The following subclauses describe components for locales themselves, the standard facets, and facilities from the ISO C library, as summarized in Table 68.

Subclause	Header(s)
22.3 Locales	<code><locale></code>
22.4 Standard locale categories	
22.5 Standard code conversion facets	<code><codecvt></code>
22.6 C library locales	<code><locale></code>

22.2 Header `<locale>` synopsis [locale.syn]

```
namespace std {
    // 22.3.1, locale
    class locale;
    template <class Facet> const Facet& use_facet(const locale&);
    template <class Facet> bool has_facet(const locale&) noexcept;

    // 22.3.3, convenience interfaces
    template <class charT> bool isspace (charT c, const locale& loc);
    template <class charT> bool isprint (charT c, const locale& loc);
    template <class charT> bool iscntrl (charT c, const locale& loc);
    template <class charT> bool isupper (charT c, const locale& loc);
    template <class charT> bool islower (charT c, const locale& loc);
    template <class charT> bool isalpha (charT c, const locale& loc);
    template <class charT> bool isdigit (charT c, const locale& loc);
    template <class charT> bool ispunct (charT c, const locale& loc);
    template <class charT> bool isxdigit(charT c, const locale& loc);
    template <class charT> bool isalnum (charT c, const locale& loc);
    template <class charT> bool isgraph (charT c, const locale& loc);
    template <class charT> bool isblank (charT c, const locale& loc);
    template <class charT> charT toupper(charT c, const locale& loc);
    template <class charT> charT tolower(charT c, const locale& loc);
    template <class Codecvt, class Elem = wchar_t,
        class Wide_alloc = allocator<Elem>,
        class Byte_alloc = allocator<char>>
        class wstring_convert;
    template <class Codecvt, class Elem = wchar_t,
        class Tr = char_traits<Elem>>
        class wbuffer_convert;

    // ...
}
```

22.3.3.2.2 Class template `wstring_convert` [conversions.string]

1. Class template `wstring_convert` performs conversions between a wide string and a byte string. It lets you specify a code conversion facet (like class template `codecvt`) to perform the conversions, without affecting any streams or locales. *Example:* If you want to use the code conversion facet `codecvt_utf8` to output to `cout` a UTF-8 multibyte sequence corresponding to a wide string, but you don't want to alter the locale for `cout`, you can write something like:

```
wstring_convert<std::codecvt_utf8<wchar_t>> myconv;
std::string mbstring = myconv.to_bytes(L"Hello\n");
std::cout << mbstring;
```

—end example—

```
namespace std {
template <class Codecvt, class Elem = wchar_t,
         class Wide_alloc = allocator<Elem>,
         class Byte_alloc = allocator<char>>
class wstring_convert {
public:
    using byte_string = basic_string<char, char_traits<char>, Byte_alloc>;
    using wide_string = basic_string<Elem, char_traits<Elem>, Wide_alloc>;
    using state_type = typename Codecvt::state_type;
    using int_type = typename wide_string::traits_type::int_type;

    explicit wstring_convert(Codecvt* pcvt = new Codecvt);
    wstring_convert(Codecvt* pcvt, state_type state);
    explicit wstring_convert(const byte_string& byte_err,
                           const wide_string& wide_err = wide_string());
    ~wstring_convert();

    wstring_convert(const wstring_convert&) = delete;
    wstring_convert& operator=(const wstring_convert&) = delete;

    wide_string from_bytes(char byte);
    wide_string from_bytes(const char* ptr);
    wide_string from_bytes(const byte_string& str);
    wide_string from_bytes(const char* first, const char* last);
    byte_string to_bytes(Elem wchar);
    byte_string to_bytes(const Elem* wptr);
    byte_string to_bytes(const wide_string& wstr);
    byte_string to_bytes(const Elem* first, const Elem* last);
    size_t converted() const noexcept;
    state_type state() const;
private:
    byte_string byte_err_string;
    wide_string wide_err_string;
    Codecvt* cvtptr;
    state_type cvtstate;
    size_t cvtcount;
};
}
```

2. The class template describes an object that controls conversions between wide string objects of class `basic_string<Elem, char_traits<Elem>, Wide_alloc>` and byte string objects of class `basic_string<char, char_traits<char>, Byte_alloc>`. The class template defines the types `wide_string` and `byte_string` as synonyms for these two types. Conversion between a sequence of `Elem` values (stored in a `wide_string` object) and multibyte sequences (stored in a `byte_string` object) is performed by an object of class `Codecvt`, which meets the requirements of the standard code-conversion facet `codecvt<Elem, char, mbstate_t>`.
3. An object of this class template stores:
 1. —`byte_err_string`— a byte string to display on errors
 2. —`wide_err_string`— a wide string to display on errors
 3. —`cvtptr`— a pointer to the allocated conversion object (which is freed when the `wstring_convert` object is destroyed)
 4. —`cvtstate`— a conversion state object
 5. —`cvtcount`— a conversion count

```
using byte_string = basic_string<char, char_traits<char>, Byte_alloc>;
```

4. The type shall be a synonym for `basic_string<char, char_traits<char>, Byte_alloc>`

```
size_t converted() const noexcept;
```

5. *Returns:* `cvtcount`;

```
wide_string from_bytes(char byte);
```

```

wide_string_from_bytes(const char* ptr);
wide_string_from_bytes(const byte_string& str);
wide_string_from_bytes(const char* first, const char* last);

```

6. *Effects:* The first member function shall convert the single-element sequence `byte` to a wide string. The second member function shall convert the null-terminated sequence beginning at `ptr` to a wide string. The third member function shall convert the sequence stored in `str` to a wide string. The fourth member function shall convert the sequence defined by the range `{first, last}` to a wide string.

7. In all cases:

1. — If the `cvtstate` object was not constructed with an explicit value, it shall be set to its default value (the initial conversion state) before the conversion begins. Otherwise it shall be left unchanged.
2. — The number of input elements successfully converted shall be stored in `cvtcount`.

8. *Returns:* If no conversion error occurs, the member function shall return the converted wide string. Otherwise, if the object was constructed with a wide-error string, the member function shall return the wide-error string. Otherwise, the member function throws an object of class `range_error`.

```

using int_type = typename wide_string::traits_type::int_type;

```

9. The type shall be a synonym for `wide_string::traits_type::int_type`.

```

state_type state() const;

```

10. *returns* `cvtstate`:

```

using state_type = typename Codecvt::state_type;

```

11. The type shall be a synonym for `Codecvt::state_type`.

```

byte_string_to_bytes(Elem wchar);
byte_string_to_bytes(const Elem* wptr);
byte_string_to_bytes(const wide_string& wstr);
byte_string_to_bytes(const Elem* first, const Elem* last);

```

12. *Effects:* The first member function shall convert the single-element sequence `wchar` to a byte string. The second member function shall convert the null-terminated sequence beginning at `wptr` to a byte string. The third member function shall convert the sequence stored in `wstr` to a byte string. The fourth member function shall convert the sequence defined by the range `{first, last}` to a byte string.

13. In all cases:

1. — If the `cvtstate` object was not constructed with an explicit value, it shall be set to its default value (the initial conversion state) before the conversion begins. Otherwise it shall be left unchanged.
2. — The number of input elements successfully converted shall be stored in `cvtcount`.

14. *Returns:* If no conversion error occurs, the member function shall return the converted byte string. Otherwise, if the object was constructed with a byte-error string, the member function shall return the byte-error string. Otherwise, the member function shall throw an object of class `range_error`.

```

using wide_string = basic_string<Elem, char_traits<Elem>, Wide_alloc>;

```

15. The type shall be a synonym for `basic_string<Elem, char_traits<Elem>, Wide_alloc>`.

```

explicit wstring_convert(Codecvt* pcvt = new Codecvt);
wstring_convert(Codecvt* pcvt, state_type state);
explicit wstring_convert(const byte_string& byte_err,
    const wide_string& wide_err = wide_string());

```

16. *Requires:* For the first and second constructors, `pcvt != nullptr`.

17. *Effects:* The first constructor shall store `pcvt` in `cvtptr` and default values in `cvtstate`, `byte_err_string`, and `wide_err_string`. The second constructor shall store `pcvt` in `cvtptr`, `state` in `cvtstate`, and default values in `byte_err_string` and `wide_err_string`; moreover the stored state shall be retained between calls to `from_bytes` and `to_bytes`. The third constructor shall store new `Codecvt` in `cvtptr`, `state_type()` in `cvtstate`, `byte_err` in `byte_err_string`, and `wide_err` in `wide_err_string`.

```

~wstring_convert();

```

18. *Effects:* The destructor shall delete `cvtptr`.

22.3.3.2.3 Class template `wbuffer_convert` [conversions.buffer]

1. Class template `wbuffer_convert` looks like a wide stream buffer, but performs all its I/O through an underlying byte stream buffer that you specify when you construct it. Like class template `wstring_convert`, it lets you specify a code conversion facet to perform the conversions, without affecting any streams or locales.

```
namespace std {
template <class Codecvt, class Elem = wchar_t, class Tr = char_traits<Elem>>
class wbuffer_convert
: public basic_streambuf<Elem, Tr> {
public:
    using state_type = typename Codecvt::state_type;

    explicit wbuffer_convert(streambuf* bytebuf = 0,
                            Codecvt* pcvt = new Codecvt,
                            state_type state = state_type());

    ~wbuffer_convert();

    wbuffer_convert(const wbuffer_convert&) = delete;
    wbuffer_convert& operator=(const wbuffer_convert&) = delete;

    streambuf* rdbuf() const;
    streambuf* rdbuf(streambuf* bytebuf);

    state_type state() const;

private:
    streambuf* bufptr; // exposition only
    Codecvt* cvtptr; // exposition only
    state_type cvtstate; // exposition only
};
}
```

2. The class template describes a stream buffer that controls the transmission of elements of type `Elem`, whose character traits are described by the class `Tr`, to and from a byte stream buffer of type `streambuf`. Conversion between a sequence of `Elem` values and multibyte sequences is performed by an object of class `Codecvt`, which shall meet the requirements of the standard code conversion facet `codecvt<Elem, char, mbstate_t>`.
3. An object of this class template stores:
 1. `bufptr` — a pointer to its underlying byte stream buffer
 2. `cvtptr` — a pointer to the allocated conversion object (which is freed when the `wbuffer_convert` object is destroyed)
 3. `cvtstate` — a conversion state object

```
state_type state() const;
```

4. *Returns:* `cvtstate`;

```
streambuf* rdbuf() const;
```

5. *Returns:* `bufptr`;

```
streambuf* rdbuf(streambuf* bytebuf);
```

6. *Effects:* Stores `bytebuf` in `bufptr`;
7. *Returns:* The previous value of `bufptr`;

```
using state_type = typename Codecvt::state_type;
```

8. The type shall be a synonym for `Codecvt::state_type`;

```
explicit wbuffer_convert(streambuf* bytebuf = 0,
                        Codecvt* pcvt = new Codecvt,
                        state_type state = state_type());
```

9. *Requires:* `pcvt != nullptr`;
10. *Effects:* The constructor constructs a stream buffer object, initializes `bufptr` to `bytebuf`, initializes `cvtptr` to `pcvt`, and initializes `cvtstate` to `state`;

```
~wbuffer_convert();
```

11. *Effects:* The destructor shall delete `cvtptr`;

22.5 Standard code conversion facets [locale.stdevt]

22.5.1 Header `<codecvt>` synopsis `[codecvt.syn]`

```
namespace std {
    enum codecvt_mode {
        consume_header = 4,
        generate_header = 2,
        little_endian = 1
    };

    template<class Elem, unsigned long Maxcode = 0x10ffff,
            codecvt_mode Mode = (codecvt_mode)0>
    class codecvt_utf8
    : public codecvt<Elem, char, mbstate_t> {
    public:
        explicit codecvt_utf8(size_t refs = 0);
        ~codecvt_utf8();
    };

    template<class Elem, unsigned long Maxcode = 0x10ffff,
            codecvt_mode Mode = (codecvt_mode)0>
    class codecvt_utf16
    : public codecvt<Elem, char, mbstate_t> {
    public:
        explicit codecvt_utf16(size_t refs = 0);
        ~codecvt_utf16();
    };

    template<class Elem, unsigned long Maxcode = 0x10ffff,
            codecvt_mode Mode = (codecvt_mode)0>
    class codecvt_utf8_utf16
    : public codecvt<Elem, char, mbstate_t> {
    public:
        explicit codecvt_utf8_utf16(size_t refs = 0);
        ~codecvt_utf8_utf16();
    };
}
```

1. The header `<codecvt>` provides code conversion facets for various character encodings.

22.5.2 Requirements `[locale.stdcvt.req]`

1. For each of the three code conversion facets `codecvt_utf8`, `codecvt_utf16`, and `codecvt_utf8_utf16`:
 1. — `Elem` is the wide-character type, such as `wchar_t`, `char16_t`, or `char32_t`.
 2. — `Maxcode` is the largest wide-character code that the facet will read or write without reporting a conversion error.
 3. — If `(Mode & consume_header)`, the facet shall consume an initial header sequence, if present, when reading a multibyte sequence to determine the endianness of the subsequent multibyte sequence to be read.
 4. — If `(Mode & generate_header)`, the facet shall generate an initial header sequence when writing a multibyte sequence to advertise the endianness of the subsequent multibyte sequence to be written.
 5. — If `(Mode & little_endian)`, the facet shall generate a multibyte sequence in little-endian order, as opposed to the default big-endian order.
2. For the facet `codecvt_utf8`:
 1. — The facet shall convert between UTF-8 multibyte sequences and UCS2 or UCS4 (depending on the size of `Elem`) within the program.
 2. — Endianness shall not affect how multibyte sequences are read or written.
 3. — The multibyte sequences may be written as either a text or a binary file.
3. For the facet `codecvt_utf16`:
 1. — The facet shall convert between UTF-16 multibyte sequences and UCS2 or UCS4 (depending on the size of `Elem`) within the program.
 2. — Multibyte sequences shall be read or written according to the `Mode` flag, as set out above.
 3. — The multibyte sequences may be written only as a binary file. Attempting to write to a text file produces undefined behavior.
4. For the facet `codecvt_utf8_utf16`:
 1. — The facet shall convert between UTF-8 multibyte sequences and UTF-16 (one or two 16-bit codes) within the program.
 2. — Endianness shall not affect how multibyte sequences are read or written.
 3. — The multibyte sequences may be written as either a text or a binary file.

See also: ISO/IEC 10646-1:1993.

D.X Standard code conversion facets `[depr.locale.stdcvt]`

1. The header `<codecvt>` provides code conversion facets for various character encodings.

D.X.1 Header `<codecvt>` synopsis [`depr.codecvt.syn`]

```
namespace std {
    enum codecvt_mode {
        consume_header = 4,
        generate_header = 2,
        little_endian = 1
    };

    template<class Elem, unsigned long Maxcode = 0x10ffff,
            codecvt_mode Mode = (codecvt_mode)0>
    class codecvt_utf8
    : public codecvt<Elem, char, mbstate_t> {
    public:
        explicit codecvt_utf8(size_t refs = 0);
        ~codecvt_utf8();
    };

    template<class Elem, unsigned long Maxcode = 0x10ffff,
            codecvt_mode Mode = (codecvt_mode)0>
    class codecvt_utf16
    : public codecvt<Elem, char, mbstate_t> {
    public:
        explicit codecvt_utf16(size_t refs = 0);
        ~codecvt_utf16();
    };

    template<class Elem, unsigned long Maxcode = 0x10ffff,
            codecvt_mode Mode = (codecvt_mode)0>
    class codecvt_utf8_utf16
    : public codecvt<Elem, char, mbstate_t> {
    public:
        explicit codecvt_utf8_utf16(size_t refs = 0);
        ~codecvt_utf8_utf16();
    };
}
```

D.X.2 Requirements [`depr.locale.stdcvt.req`]

1. For each of the three code conversion facets `codecvt_utf8`, `codecvt_utf16`, and `codecvt_utf8_utf16`:
 1. — `Elem` is the wide-character type, such as `wchar_t`, `char16_t`, or `char32_t`.
 2. — `Maxcode` is the largest wide-character code that the facet will read or write without reporting a conversion error.
 3. — If `(Mode & consume_header)`, the facet shall consume an initial header sequence, if present, when reading a multibyte sequence to determine the endianness of the subsequent multibyte sequence to be read.
 4. — If `(Mode & generate_header)`, the facet shall generate an initial header sequence when writing a multibyte sequence to advertise the endianness of the subsequent multibyte sequence to be written.
 5. — If `(Mode & little_endian)`, the facet shall generate a multibyte sequence in little-endian order, as opposed to the default big-endian order.
2. For the facet `codecvt_utf8`:
 1. — The facet shall convert between UTF-8 multibyte sequences and UCS2 or UCS4 (depending on the size of `Elem`) within the program.
 2. — Endianness shall not affect how multibyte sequences are read or written.
 3. — The multibyte sequences may be written as either a text or a binary file.
3. For the facet `codecvt_utf16`:
 1. — The facet shall convert between UTF-16 multibyte sequences and UCS2 or UCS4 (depending on the size of `Elem`) within the program.
 2. — Multibyte sequences shall be read or written according to the `Mode` flag, as set out above.
 3. — The multibyte sequences may be written only as a binary file. Attempting to write to a text file produces undefined behavior.
4. For the facet `codecvt_utf8_utf16`:
 1. — The facet shall convert between UTF-8 multibyte sequences and UTF-16 (one or two 16-bit codes) within the program.
 2. — Endianness shall not affect how multibyte sequences are read or written.
 3. — The multibyte sequences may be written as either a text or a binary file.

See also: ISO/IEC 10646-1:1993.

D.Y Deprecated Character Conversions [depr.conversions]

The header `<locale>` has the following additions:

```
template<class Codecvt, class Elem = wchar_t,
        class Wide_alloc = allocator<Elem>,
        class Byte_alloc = allocator<char>>
class wstring_convert;
template<class Codecvt, class Elem = wchar_t,
        class Tr = char_traits<Elem>>
class wbuffer_convert;
```

D.Y.1 Class template wstring_convert [depr.conversions.string]

1. Class template `wstring_convert` performs conversions between a wide string and a byte string. It lets you specify a code conversion facet (like class template `codecvt`) to perform the conversions, without affecting any streams or locales. [*Example:* If you want to use the code conversion facet `codecvt_utf8` to output to `cout` a UTF-8 multibyte sequence corresponding to a wide string, but you don't want to alter the locale for `cout`, you can write something like:

```
wstring_convert<std::codecvt_utf8<wchar_t>> myconv;
std::string mbstring = myconv.to_bytes(L"Hello\n");
std::cout << mbstring;
```

— end example]

```
namespace std {
template <class Codecvt, class Elem = wchar_t,
        class Wide_alloc = allocator<Elem>,
        class Byte_alloc = allocator<char>>
class wstring_convert {
public:
    using byte_string = basic_string<char, char_traits<char>, Byte_alloc>;
    using wide_string = basic_string<Elem, char_traits<Elem>, Wide_alloc>;
    using state_type = typename Codecvt::state_type;
    using int_type = typename wide_string::traits_type::int_type;

    explicit wstring_convert(Codecvt* pcvt = new Codecvt);
    wstring_convert(Codecvt* pcvt, state_type state);
    explicit wstring_convert(const byte_string& byte_err,
                           const wide_string& wide_err = wide_string());
    ~wstring_convert();

    wstring_convert(const wstring_convert&) = delete;
    wstring_convert& operator=(const wstring_convert&) = delete;

    wide_string from_bytes(char byte);
    wide_string from_bytes(const char* ptr);
    wide_string from_bytes(const byte_string& str);
    wide_string from_bytes(const char* first, const char* last);
    byte_string to_bytes(Elem wchar);
    byte_string to_bytes(const Elem* wptr);
    byte_string to_bytes(const wide_string& wstr);
    byte_string to_bytes(const Elem* first, const Elem* last);
    size_t converted() const noexcept;
    state_type state() const;
private:
    byte_string byte_err_string;
    wide_string wide_err_string;
    Codecvt* cvtptr;
    state_type cvtstate;
    size_t cvtcount;
};
}
```

2. The class template describes an object that controls conversions between wide string objects of class `basic_string<Elem, char_traits<Elem>, Wide_alloc>` and byte string objects of class `basic_string<char, char_traits<char>, Byte_alloc>`. The class template defines the types `wide_string` and `byte_string` as synonyms for these two types. Conversion between a sequence of `Elem` values (stored in a `wide_string` object) and multibyte sequences (stored in a `byte_string` object) is performed by an object of class `Codecvt`, which meets the requirements of the standard code-conversion facet `codecvt<Elem, char, mbstate_t>`.
3. An object of this class template stores:
 1. — `byte_err_string` — a byte string to display on errors
 2. — `wide_err_string` — a wide string to display on errors
 3. — `cvtptr` — a pointer to the allocated conversion object (which is freed when the `wstring_convert` object is destroyed)

4. `— cvtstate — a conversion state object`
5. `— cvtcount — a conversion count`

```
using byte_string = basic_string<char, char_traits<char>, Byte_alloc>;
```

4. The type shall be a synonym for `basic_string<char, char_traits<char>, Byte_alloc>`

```
size_t converted() const noexcept;
```

5. *Returns:* `cvtcount`.

```
wide_string from_bytes(char byte);  
wide_string from_bytes(const char* ptr);  
wide_string from_bytes(const byte_string& str);  
wide_string from_bytes(const char* first, const char* last);
```

6. *Effects:* The first member function shall convert the single-element sequence `byte` to a wide string. The second member function shall convert the null-terminated sequence beginning at `ptr` to a wide string. The third member function shall convert the sequence stored in `str` to a wide string. The fourth member function shall convert the sequence defined by the range `[first, last)` to a wide string.

7. In all cases:

1. — If the `cvtstate` object was not constructed with an explicit value, it shall be set to its default value (the initial conversion state) before the conversion begins. Otherwise it shall be left unchanged.
2. — The number of input elements successfully converted shall be stored in `cvtcount`.

8. *Returns:* If no conversion error occurs, the member function shall return the converted wide string. Otherwise, if the object was constructed with a wide-error string, the member function shall return the wide-error string. Otherwise, the member function throws an object of class `range_error`.

```
using int_type = typename wide_string::traits_type::int_type;
```

9. The type shall be a synonym for `wide_string::traits_type::int_type`.

```
state_type state() const;
```

10. *returns* `cvtstate`.

```
using state_type = typename Codecvt::state_type;
```

11. The type shall be a synonym for `Codecvt::state_type`.

```
byte_string to_bytes(Elem wchar);  
byte_string to_bytes(const Elem* wptr);  
byte_string to_bytes(const wide_string& wstr);  
byte_string to_bytes(const Elem* first, const Elem* last);
```

12. *Effects:* The first member function shall convert the single-element sequence `wchar` to a byte string. The second member function shall convert the null-terminated sequence beginning at `wptr` to a byte string. The third member function shall convert the sequence stored in `wstr` to a byte string. The fourth member function shall convert the sequence defined by the range `[first, last)` to a byte string.

13. In all cases:

1. — If the `cvtstate` object was not constructed with an explicit value, it shall be set to its default value (the initial conversion state) before the conversion begins. Otherwise it shall be left unchanged.
2. — The number of input elements successfully converted shall be stored in `cvtcount`.

14. *Returns:* If no conversion error occurs, the member function shall return the converted byte string. Otherwise, if the object was constructed with a byte-error string, the member function shall return the byte-error string. Otherwise, the member function shall throw an object of class `range_error`.

```
using wide_string = basic_string<Elem, char_traits<Elem>, Wide_alloc>;
```

15. The type shall be a synonym for `basic_string<Elem, char_traits<Elem>, Wide_alloc>`.

```
explicit wstring_convert(Codecvt* pcvt = new Codecvt);  
wstring_convert(Codecvt* pcvt, state_type state);  
explicit wstring_convert(const byte_string& byte_err,  
                        const wide_string& wide_err = wide_string());
```

16. *Requires:* For the first and second constructors, `pcvt != nullptr`.
17. *Effects:* The first constructor shall store `pcvt` in `cvtptr` and default values in `cvtstate`, `byte_err_string`, and `wide_err_string`. The second constructor shall store `pcvt` in `cvtptr`, state in `cvtstate`, and default values in `byte_err_string` and `wide_err_string`; moreover the stored state shall be retained between calls to `from_bytes` and to `bytes`. The third constructor shall store new `Codecvt` in `cvtptr`, state `type()` in `cvtstate`, `byte_err` in `byte_err_string`, and `wide_err` in `wide_err_string`.

```
~wstring_convert();
```

18. *Effects:* The destructor shall delete `cvtptr`.

D.Y.2 Class template `wbuffer_convert` [depr.conversions.buffer]

1. Class template `wbuffer_convert` looks like a wide stream buffer, but performs all its I/O through an underlying byte stream buffer that you specify when you construct it. Like class template `wstring_convert`, it lets you specify a code conversion facet to perform the conversions, without affecting any streams or locales.

```
namespace std {
template <class Codecvt, class Elem = wchar_t, class Tr = char_traits<Elem>>
class wbuffer_convert
: public basic_streambuf<Elem, Tr> {
public:
    using state_type = typename Codecvt::state_type;

    explicit wbuffer_convert(streambuf* bytebuf = 0,
                           Codecvt* pcvt = new Codecvt,
                           state_type state = state_type());

    ~wbuffer_convert();

    wbuffer_convert(const wbuffer_convert&) = delete;
    wbuffer_convert& operator=(const wbuffer_convert&) = delete;

    streambuf* rdbuf() const;
    streambuf* rdbuf(streambuf* bytebuf);

    state_type state() const;

private:
    streambuf* bufptr;           // exposition only
    Codecvt* cvtptr;            // exposition only
    state_type cvtstate;        // exposition only
};
}
```

2. The class template describes a stream buffer that controls the transmission of elements of type `Elem`, whose character traits are described by the class `Tr`, to and from a byte stream buffer of type `streambuf`. Conversion between a sequence of `Elem` values and multibyte sequences is performed by an object of class `Codecvt`, which shall meet the requirements of the standard code-conversion facet `codecvt<Elem, char, mbstate_t>`.
3. An object of this class template stores:
 1. `— bufptr` — a pointer to its underlying byte stream buffer
 2. `— cvtptr` — a pointer to the allocated conversion object (which is freed when the `wbuffer_convert` object is destroyed)
 3. `— cvtstate` — a conversion state object

```
state_type state() const;
```

4. *Returns:* `cvtstate`.

```
streambuf* rdbuf() const;
```

5. *Returns:* `bufptr`.

```
streambuf* rdbuf(streambuf* bytebuf);
```

6. *Effects:* Stores `bytebuf` in `bufptr`.
7. *Returns:* The previous value of `bufptr`.

```
using state_type = typename Codecvt::state_type;
```

8. The type shall be a synonym for `Codecvt::state_type`.

```
explicit wbuffer convert(streambuf* bytebuf = 0,  
    Codecvt* pcvt = new Codecvt, state_type state = state_type());
```

9. *Requires:* `pcvt != nullptr`.
10. *Effects:* The constructor constructs a stream buffer object, initializes `bufptr` to `bytebuf`, initializes `cvtptr` to `pcvt`, and initializes `cvtstate` to `state`.

```
~wbuffer convert();
```

11. *Effects:* The destructor shall delete `cvtptr`.